

Engenharia de Software

Processos

Prof. Tiago Piperno Bonetti

Material adaptado de: Marco Tulio Valente
<https://engsoftmoderna.info>

XP =
Valores + Princípios + Práticas

Práticas

- ✓ Processo de Desenvolvimento
- ✓ Programação
- ✓ Gerenciamento de Projetos

Processo de desenvolvimento

- » Representante dos Clientes
- » Histórias de Usuário
- » Iterações
- » Releases
- » Planejamento de Releases
- » Planejamento de Iterações
- » Slack

Processo de Desenvolvimento

Envolvimento dos **clientes** com o projeto.

- » Os times incluem pelo menos um **representante dos clientes**, que deve entender do domínio do sistema que será construído.
- » Função: **escrever as histórias de usuário** (user stories) - documentos que descrevem os **requisitos** do sistema.
- » Histórias são **documentos resumidos e simples**.
- » O representante dos clientes define o que ele **deseja que o sistema faça**, usando sua própria linguagem.

Processo de Desenvolvimento

Exemplo: história de um sistema de perguntas e respostas — semelhante ao famoso Stack Overflow.

Postar Pergunta

Um usuário, quando logado no sistema, deve ser capaz de postar perguntas. Como é um site sobre programação, as perguntas podem incluir blocos de código, os quais devem ser apresentados com um leiaute diferenciado.

A história tem um título (Postar Pergunta) e uma breve descrição, que não ocupa mais do que duas ou três sentenças.

Processo de Desenvolvimento

Depois de escritas as histórias são **estimadas pelos desenvolvedores**.

- » Definem, mesmo que preliminarmente, quanto **tempo** será necessário para **implementar**.
- » Frequentemente, a duração de uma história é estimada em **story points**, em vez de horas ou homens/hora.
- » As histórias **mais simples** são estimadas como tendo tamanho igual a **1 story point**;
- » Histórias que são cerca de **duas vezes mais complexas** do que as primeiras são estimadas como tendo **2 story points** e assim por diante.

Processo de Desenvolvimento

Depois de escritas as histórias são **estimadas pelos desenvolvedores**.

- » Muitas vezes, usa-se uma **sequência de Fibonacci** para definir a **escala** de possíveis story points, como em 1, 2, 3, 5, 8, 13 story points.

*O objetivo é criar uma escala que torne as tarefas progressivamente mais difíceis e, ao mesmo tempo, permita ao time realizar **comparações similares à seguinte**:*

- será que o esforço para implementar essa tarefa que planejamos estimar com 8 story points é equivalente ao esforço de implementar uma tarefa na escala anterior (5 story points) e mais uma tarefa na próxima escala inferior (3 pontos)?

Se sim, 8 story points é uma boa estimativa. Senão, o melhor é estimar a história com 5 story points.

Processo de Desenvolvimento

A **implementação** das histórias ocorre em **iterações**, as quais têm uma **duração fixa e bem definida**, variando de **uma a três semanas**, por exemplo.

As iterações, por sua vez, formam **ciclos mais longos**, chamados de **releases**, de **dois a três meses**, por exemplo.

Sugere-se que o representante dos clientes escreva **histórias** que **requeiram** pelo menos **uma release** para serem implementadas.

» O horizonte de **planejamento** é uma **release**, isto é, alguns meses.

Aviso: Em XP, a palavra release tem um sentido diferente daquele que se usa em gerência de configuração. Em gerência de configuração, uma release é uma versão de um sistema que será disponibilizada para seus usuários finais. Não necessariamente a versão do sistema ao final de uma release de XP precisa entrar em produção.

Processo de Desenvolvimento

Em **resumo**, para **começar** a usar XP precisamos de:

- » Definir a **duração** de uma **iteração**.
- » Definir o **número** de **iterações** de uma **release**.
- » Um **conjunto** de **histórias**, escritas pelo representante dos clientes.
- » **Estimativas** para cada história, feitas pelos desenvolvedores.
- » Definir a **velocidade** do **time**, isto é, o número de story points que ele consegue implementar por iteração.

Processo de Desenvolvimento

Próxima etapa: o representante do cliente deve **priorizar as histórias**.

- » Definir **quais histórias serão implementadas** nas iterações da primeira release.
- » Deve-se respeitar a velocidade do time de desenvolvimento.

Exemplo: suponha que a velocidade de um time seja de 25 story points por iteração.

Nesse caso, o representante do cliente não pode alocar histórias para uma iteração cujo somatório de story points ultrapasse esse limite.

A tarefa de alocar histórias a iterações e releases é chamada de planejamento de releases (ou então planning game, que foi o nome adotado na primeira edição do livro de XP).

Exemplo

Fórum de perguntas e respostas.

Resultado de um possível planejamento de releases.

Representante dos clientes escreveu 8 histórias.

Cada release possui duas iterações.

A velocidade do time é de 21 story points por iteração (somatório dos story points de cada iteração é exatamente igual a 21).

História	Story Points	Iteração	Release
Cadastrar usuário	8	1	1
Postar perguntas	5	1	1
Postar respostas	3	1	1
Tela de abertura	5	1	1
Gamificar perguntas/respostas	5	2	1
Pesquisar perguntas/respostas	8	2	1
Adicionar tags	5	2	1
Comentar perguntas/respostas	3	2	1

Processo de Desenvolvimento

Próximo passo: **começar as iterações**.

O time de desenvolvimento deve se reunir para realizar o **planejamento da iteração**.

- » Objetivo: **decompor** as histórias de uma iteração em **tarefas**.
- » Devem corresponder a **atividades de programação** que possam ser **alocadas** para um dos **desenvolvedores** do time.

Por exemplo, a seguinte lista mostra as tarefas para a história Postar Perguntas, que é a primeira história que será implementada em nosso sistema de exemplo.

Processo de Desenvolvimento

Exemplo: a seguinte lista mostra as **tarefas para a história Postar Perguntas**, que é a primeira história que será implementada.

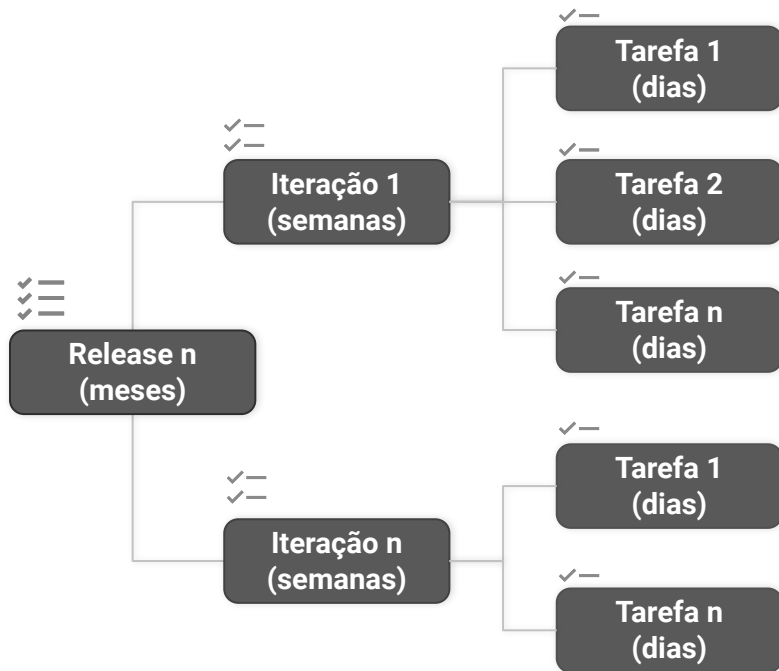
- » Projetar e testar a interface Web, incluindo leiaute, CSS templates, etc.
- » Instalar banco de dados, projetar e criar tabelas.
- » Implementar a camada de acesso a dados.
- » Instalar servidor e testar framework web.
- » Implementar camada de controle, com operações para cadastrar, remover e atualizar perguntas.
- » Implementar interface Web.

Como regra geral, as tarefas não devem ser complexas, devendo ser possível concluí-las em alguns dias.

Processo de Desenvolvimento

Resumindo, um **projeto** XP é organizado em:

- » **Releases**, que são conjunto de iterações, com duração total de alguns meses.
- » **Iterações**, que são conjuntos de tarefas resultantes da decomposição de histórias, com duração total de algumas semanas.
- » **Tarefas**, com duração de alguns dias.



Processo de Desenvolvimento

Uma **iteração termina** quando todas as **histórias** estiverem **implementadas** e **validadas** pelo representante dos clientes.

- » O representante dos clientes deve **concordar** que elas **atendem** ao que ele **especificou**.

XP defende que os times **programem algumas folgas** (slacks), que são **tarefas** que podem ser **adiadas**, caso necessário.

- » Exemplo: o estudo de uma nova tecnologia, a realização de um curso online, preparar uma documentação ou desenvolver um projeto paralelo.
- » Google, por exemplo, permite que seus desenvolvedores usem 20% de seu tempo para desenvolver um projeto pessoal.

Processo de Desenvolvimento

Folgas têm dois objetivos principais:

- 1) criar um **buffer de segurança** em uma iteração, que possa ser usado caso alguma tarefa demande mais tempo do que o previsto;
- 2) permitir que os **desenvolvedores respirem um pouco**, pois o ritmo de trabalho em projetos de desenvolvimento de software costuma ser intenso e desgastante.

Logo, os desenvolvedores precisam de um tempo para realizarem algumas tarefas sem que exista uma cobrança imediata de resultados.

Práticas de Programação

- » Design incremental
- » Programação em pares
- » Propriedade coletiva de código
- » Testes Automatizados
- » Desenvolvimento Dirigido Por Testes (TDD)
- » Build Automatizado
- » Integração Contínua

Práticas de Programação

O nome Extreme Programming foi escolhido porque XP propõe um **conjunto de práticas de programação inovadoras**.

- » Programação em pares, testes automatizados, desenvolvimento dirigido por testes (TDD), builds automatizados, integração contínua, etc.

A maioria dessas práticas passou a ser **largamente adotada pela indústria** de software e hoje são **praticamente obrigatórias** na maioria dos projetos — mesmo naqueles que não usam um método ágil.

Design Incremental

Em XP **não** há uma **fase** de design e análise **detalhados**.

A ideia é que o time deve **reservar tempo para definir o design** do sistema que está sendo desenvolvido.

- » Porém, isso deve ser uma atividade **contínua e incremental**.
- » Simplicidade é o valor de XP que norteia essa opção.

Quando o design é confinado no início do projeto, correm-se diversos **riscos**.

- » Os requisitos ainda não estão totalmente claros para o time
- » Nem mesmo para o representante dos clientes.

Design Incremental

Exemplo:

- » Pode-se **supervalorizar** alguns **requisitos**, que mais tarde irão se revelar **menos importantes**;
- » Pode-se **subvalorizar** outros **requisitos**, que depois, com o decorrer da implementação, irão assumir um maior **protagonismo**.
- » **Novos requisitos** podem surgir ao longo do projeto, tornando o design inicial desatualizado e menos eficiente.

Design Incremental

Observações importantes:

- 1) Times **experientes** costumam ter uma **boa aproximação** do design logo na primeira iteração.
 - » Exemplo: eles já sabem que se trata de um sistema com interface Web, com uma camada de lógica não tão complexa, e depois com uma camada de persistência e um banco de dados, certamente relacional.
- 2) Nada impede que na primeira iteração o time crie uma **tarefa técnica** para **discutir e refinar** o **design** que vão começar a adotar no sistema.
- 3) XP defende que **refactoring** deve ser continuamente aplicado para melhorar a qualidade do design.

Programação em Pares (Pair Programming)

Apesar de polêmica, a **ideia é simples**:

- » Toda tarefa de codificação — incluindo implementação de uma nova história, de um teste, ou a correção de um bug — deve ser realizada por **dois desenvolvedores trabalhando juntos**, compartilhando o mesmo teclado e monitor.



Programação em Pares (Pair Programming)

- » Um dos desenvolvedores é o **líder** (ou driver) da sessão, ficando com o **teclado** e o **mouse**.
- » Ao segundo desenvolvedor cabe a função de **revisor** e **questionador**, no bom sentido.
 - Esse segundo desenvolvedor é chamado de **navegador**.
 - O nome vem dos ralis automobilísticos, nos quais os pilotos são acompanhados de um navegador.



Programação em Pares (Pair Programming)

Por que adotar?

Espera-se **melhorar** a **qualidade** do **código** e do **design**, pois duas cabeças pensam melhor do que uma.

Contribui para **disseminar** o **conhecimento** sobre o **código**, que não fica nas mãos e na cabeça de apenas um desenvolvedor.

- » Determinado desenvolvedor tem dificuldade para sair de férias, pois apenas ele conhece uma parte crítica do código.

Pode ser usada para **treinar** desenvolvedores **menos experientes**.

Programação em Pares (Pair Programming)

Por que NÃO adotar?

Existem **custos econômicos**, já que dois programadores estão sendo alocados para realizar uma tarefa que, a princípio, poderia ser realizada por apenas um.

Muitos **desenvolvedores** se sentem **desconfortáveis** com a prática (emocional e cognitiva).

Para aliviar esse desconforto, XP propõe que os pares sejam **trocados** a cada **sessão**.

- » Exemplo: sessão de 50 minutos, seguidos de uma pausa de 10 minutos para descanso.
- » Na próxima sessão, os **pares e papéis (líder vs revisor) são trocados**.

Programação em Pares (Pair Programming)

Mundo real: Estudo com Engenheiros da Microsoft (2008 - [link](#))

» Vantagens:

- Redução de bugs
- Código de melhor qualidade
- Disseminação de conhecimento
- Aprendizado com os pares

» Desvantagem:

- Custo

Mais recentemente, diversas empresas passaram a adotar a prática de revisão de código. A ideia é que todo código produzido por um desenvolvedor tem que ser revisado e comentado por um outro desenvolvedor, porém de forma offline e assíncrona. Ou seja, nesses casos, o revisor não estará fisicamente ao lado do líder.

Propriedade Coletiva do Código

Qualquer desenvolvedor — ou par de desenvolvedores trabalhando junto — pode modificar **qualquer parte do código**.

- » Seja para implementar uma nova feature, para corrigir um bug ou para aplicar um refactoring.

Exemplo:

- » Se você descobriu um bug em algum ponto do código, vá em frente e **corrija-o**.
- » Você **não** precisa de **autorização** de quem implementou esse código ou de quem realizou a última manutenção nele.

Testes Automatizados

Essa é uma das práticas de XP que alcançou **maior sucesso**.

Testes manuais é um procedimento **custoso** e que não pode ser reproduzido a todo momento.

XP propõe a implementação de programas — chamados de testes — para **executar pequenas unidades** de um sistema e verificar se as **saídas** produzidas são aquelas **esperadas**.

Mais ou menos na mesma época foram desenvolvidos os primeiros **frameworks** de **testes de unidade** — como o JUnit (implementado por Kent Beck e Erich Gamma).

Desenvolvimento Dirigido por Testes (TDD)

Se todo método deve possuir testes, **por que não escrevê-los primeiro?**

TDD possui duas **motivações** principais:

- 1) evitar que a escrita de testes seja sempre **deixada para amanhã**, pois eles são a primeira coisa que se deve implementar;
- 2) ao escrever um teste, o desenvolvedor se **coloca no papel de cliente** do método testado, isto é, ele primeiro em **como** eles devem **usá-lo**.

Com isso, incentiva-se a criação de métodos mais amigáveis, do ponto de vista da interface provida para os clientes.

Build Automatizado

Build é o nome que se dá para a geração de uma **versão** de um **sistema** que seja **executável** e que possa ser colocada em **produção**.

Para automatizar esse processo, são usadas **ferramentas**, como o sistema Make, Ant, Maven, Gradle, Rake, MSBuild, etc.

1) XP defende que o processo de build seja automatizado, **sem nenhuma intervenção dos desenvolvedores**.

» Assim, eles podem focar apenas na implementação de histórias.

2) XP defende que o processo de build seja o mais **rápido possível**, para que os desenvolvedores recebam rapidamente **feedback** sobre possíveis problemas.

Integração Contínua

Sistemas de software são desenvolvidos com o apoio de **sistemas de controle de versões**. Exemplo: git.

Desenvolvedores têm que **primeiro baixar (pull) o código fonte** para sua máquina local, antes de começar a trabalhar em uma tarefa.

Feito isso, eles devem **subir o código modificado (push)**.

Possível conflito: entre um pull e um push outro desenvolvedor pode ter modificado o mesmo trecho de código.

- » O sistema de controle de versão irá impedir a integração, dizendo que existe um conflito.
- » Conflitos são ruins, porque devem ser **resolvidos manualmente**.

Integração Contínua

Evitar completamente conflitos é **impossível**.

Podemos pelo menos tentar **diminuir** o **número** e o **tamanho** dos conflitos.

A ideia de XP é simples: desenvolvedores devem **integrar seu código sempre**, se possível **todos os dias**.

O objetivo é **evitar** que desenvolvedores **passem muito tempo** trabalhando **localmente**, em suas tarefas, sem integrar o código.

Integração Contínua

Costuma-se usar também um **serviço** de **integração contínua**.

Antes de realizar qualquer integração, esse serviço **faz o build do código e executa os testes**.

Existem diversos serviços de integração contínua, como Jenkins, TravisCI, CircleCI, etc.

*Por **exemplo**, quando se desenvolve um sistema usando o **GitHub**, pode-se ativar esses serviços na forma de **plugins**.*

Se o repositório GitHub for público, o serviço de integração contínua é gratuito; se for privado, deve-se pagar uma assinatura.

Práticas de Gerenciamento de Projetos

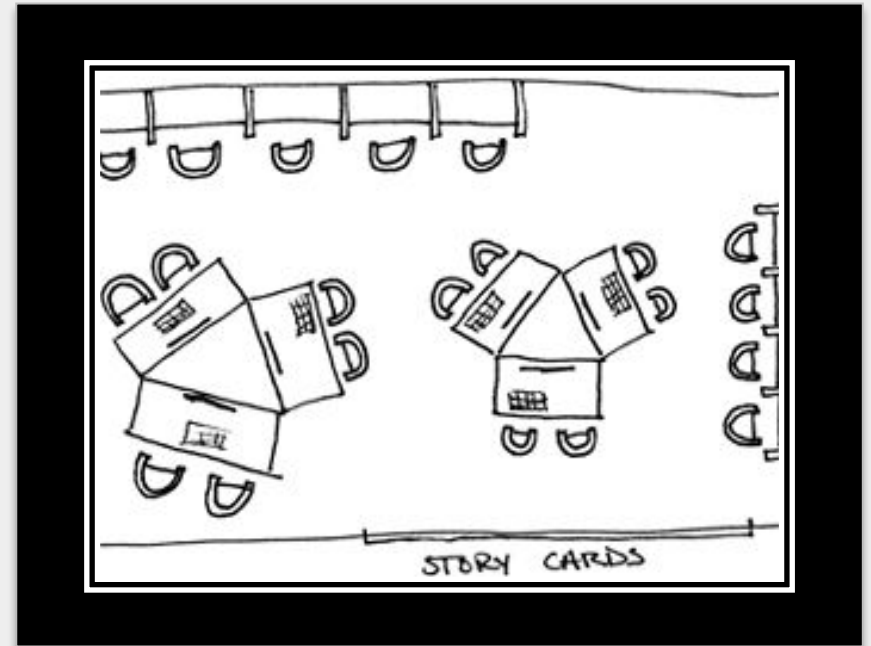
- » Ambiente de Trabalho
- » Contratos com Escopo Aberto
- » Métricas de Processo

Ambiente de Trabalho

Time pequeno, com menos de 10 desenvolvedores, por exemplo.

Todos eles devem estar **dedicados exclusivamente** ao **projeto**.

Todos os desenvolvedores trabalhem em uma **mesma sala**, para facilitar comunicação e feedback.

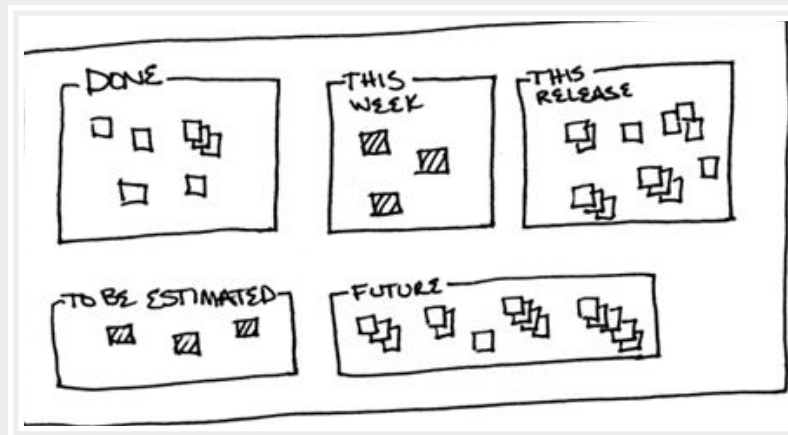


Ambiente de Trabalho

Espaço de trabalho **informativo**.

Exemplo: fixados **cartazes** nas paredes, com as **histórias** da iteração, incluindo seu **estado**: histórias pendentes, histórias em andamento e histórias concluídas.

Permitir que o time possa **visualizar o trabalho** que está sendo realizado.



Ambiente de Trabalho

Jornadas de trabalho **sustentáveis**.

Empresas de desenvolvimento de software são conhecidas por exigirem longas jornadas de trabalho, com diversas **horas extras** e trabalho nos **finais de semana**.

XP defende que essa prática **não é sustentável** e que as jornadas de trabalho devem ser sempre próximas de 40 horas, mesmo na véspera de entregas.

- » Podem causar danos à **saúde física** e **mental** dos desenvolvedores
- » Incentivar a **rotatividade** do **time**, pois os membros vão estar sempre pensando em um novo emprego.

Contratos com Escopo Aberto

Escopo fechado

- » Cliente define requisitos ("fecha escopo")
- » Empresa desenvolvedora: preço + prazo

XP advoga que esses contratos são **arriscados**.

Os requisitos mudam e nem mesmo o cliente sabe antecipadamente o que ele quer que o sistema faça, de modo preciso.

Contratos com Escopo Aberto

Escopo aberto

- » Escopo definido a cada iteração
- » Pagamento por homem/hora
 - Exemplo: alocar um time com um certo **número** de **desenvolvedores** para trabalhar integralmente no projeto. Combina-se um **preço** para a **hora** de **cada desenvolvedor**.
- » Contrato renovado a cada iteração
 - Dá ao cliente a liberdade de mudar de empresa caso não esteja satisfeito com a qualidade do serviço prestado

"colaboração com o cliente, mais do que negociação de contratos"

Contratos com Escopo Aberto

Escopo aberto

- » Escopo definido a cada iteração
- » Pagamento por homem/hora
 - Exemplo: alocar um time com um certo **número** de **desenvolvedores** para trabalhar integralmente no projeto. Combina-se um **preço** para a **hora** de **cada desenvolvedor**.
- » Contrato renovado a cada iteração
 - Dá ao cliente a liberdade de mudar de empresa caso não esteja satisfeito com a qualidade do serviço prestado

"colaboração com o cliente, mais do que negociação de contratos"

Métricas de Processo

Para que gerentes e executivos possam acompanhar um projeto XP recomenda-se o uso de **duas métricas principais**:

- 1) Número de bugs em produção (que deve ser idealmente da ordem de poucos bugs por ano).
- 2) Intervalo de tempo entre o início do desenvolvimento e o momento em que o projeto começar a gerar os seus primeiros resultados financeiros (que também deve ser pequeno, da ordem de um ano, por exemplo).